

Final Mock Exam — Solutions

Lectures 1–12 (comprehensive)

Prof. Dr. Christoph Weisser

Quantitative Research Methods · HSBI

Summer Semester 2026

Overview of the final mock exam

Problem	Topic	Points
P1	Moving beyond linearity — splines (Ch. 7)	16
P2	Generalised additive models (Ch. 7)	8
P3	Regression and classification trees by hand (Ch. 8)	20
P4	Ensembles — bagging, random forests, boosting (Ch. 8)	16
P5	Neural networks by hand (Ch. 10)	20
P6	Multiple testing (Ch. 13)	20
P7	Cumulative essentials (Ch. 2–6)	20
Σ		120

Exam conditions

120 minutes for 120 points (≈ 1 point per minute). Allowed aids: non-programmable calculator and one handwritten A4 sheet (both sides). All required constants (e-powers and logs) are given in the problems. Justify every answer.

Problem 1 — task

Problem 1 (16 P) — Moving beyond linearity

- (a) [4 P] Degrees of freedom of a cubic spline with K interior knots — justify by counting parameters and constraints; evaluate for $K = 4$.
- (b) [4 P] Degrees of freedom of a *natural* cubic spline with K knots ($K = 4$); which extra constraint acts where and why is it useful?
- (c) [4 P] Smoothing spline $\sum_i (y_i - g(x_i))^2 + \lambda \int g''(t)^2 dt$: role of the two terms; behaviour of $\hat{g}$ as $\lambda \rightarrow 0$ and $\lambda \rightarrow \infty$.
- (d) [4 P] Truncated-power basis of a cubic spline with one knot ξ ; define $(x - \xi)_+^3$; why does adding it keep the fit smooth at ξ ?

Lecture slides: Chapter 7 — Moving Beyond Linearity (Lecture 9).

Problem 1 — solution (a): cubic-spline df

Solution P1 — (a) degrees of freedom of a cubic spline

Strategy: count parameters, then subtract constraints.

- K interior knots split the range of X into $K + 1$ intervals; each interval carries its own cubic polynomial with 4 coefficients:

$$4(K + 1) = 4K + 4 \quad \text{free parameters (unconstrained pieces).}$$

- At *each* knot the neighbouring pieces must agree in value, slope and curvature: continuity of $g, g', g'' \Rightarrow 3$ constraints per knot, $3K$ in total.

$$\text{df} = \underbrace{4(K + 1)}_{\text{parameters}} - \underbrace{3K}_{\text{constraints}} = K + 4 \quad \Rightarrow \quad K = 4 : \text{df} = 4 + 4 = 8.$$

Basis check. The same space is spanned by $1, x, x^2, x^3$ plus one truncated-power term per knot, i.e. $4 + K$ basis functions — an identical count.

Problem 1 — solution (b): natural cubic spline

Solution P1 — (b) the natural cubic spline

A *natural* cubic spline adds one requirement: it must be *linear beyond the two boundary knots*, i.e. in the outer regions $X < \xi_1$ and $X > \xi_K$.

Cost of the constraint. Linearity forces the quadratic and cubic terms to vanish in each outer region $\Rightarrow$ 2 fewer parameters per boundary region, 4 in total:

$$\text{df} = (K + 4) - 4 = K \quad \Rightarrow \quad K = 4 : \text{df} = 4.$$

Where it acts: only in the two tails, outside the outermost knots.

Why it is useful

Unconstrained splines and polynomials wiggle wildly near the boundaries, where data are sparse; forcing linearity there cuts the prediction *variance* at and beyond the boundary for only a little extra bias.

Problem 1 — solution (c): smoothing spline

Solution P1 — (c) the two terms and the limits of λ

$$\min_g \underbrace{\sum_{i=1}^n (y_i - g(x_i))^2}_{\text{fit to the data (RSS)}} + \lambda \underbrace{\int g''(t)^2 dt}_{\text{roughness / curvature}} .$$

g'' measures curvature, so the integral penalises wiggleness; the tuning parameter $\lambda \geq 0$ trades the two terms off.

- $\lambda \rightarrow 0$: the penalty vanishes $\Rightarrow \hat{g}$ *interpolates* every training point — maximally wiggly, zero training RSS, severe overfitting (effective df $\rightarrow n$).
- $\lambda \rightarrow \infty$: any curvature is infinitely expensive $\Rightarrow g'' \equiv 0 \Rightarrow \hat{g}$ is the *least-squares straight line* (effective df $\rightarrow 2$).

In between, λ (chosen e.g. by LOOCV) moves the effective df smoothly from n to 2.

Problem 1 — solution (d): truncated-power basis

Solution P1 — (d) truncated-power basis with one knot ξ

A cubic spline with a single knot ξ can be written with a five-function basis:

$$f(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \beta_4 (x - \xi)_+^3, \quad (x - \xi)_+^3 = \begin{cases} (x - \xi)^3, & x > \xi \\ 0, & x \leq \xi. \end{cases}$$

Why smoothness survives. At the knot $x = \xi$ the added function and its first two derivatives all vanish:

$$(x - \xi)_+^3|_{\xi} = 0, \quad \frac{d}{dx}(x - \xi)_+^3|_{\xi} = 0, \quad \frac{d^2}{dx^2}(x - \xi)_+^3|_{\xi} = 0,$$

so adding it changes only the *third* derivative at ξ . The fit therefore stays continuous with continuous g' and g'' — exactly the defining smoothness of a cubic spline.

Problem 2 — task

Problem 2 (8 P) — Generalised additive models

- (a) [3 P] General form of a GAM for quantitative Y ; how does it extend multiple linear regression?
- (b) [3 P] One advantage concerning the interpretation of the f_j ; the main structural limitation — and its repair.
- (c) [2 P] Backfitting in one or two sentences.

Lecture slides: Chapter 7 — Moving Beyond Linearity (Lecture 9).

Problem 2 — solution (a): the GAM model form

Solution P2 — (a) general form of a GAM

$$Y = \beta_0 + f_1(X_1) + f_2(X_2) + \cdots + f_p(X_p) + \varepsilon.$$

Each predictor enters through its *own* smooth function f_j (a natural or smoothing spline, or local regression), and the contributions are *added*.

Extension of linear regression. Multiple linear regression is the special case $f_j(X_j) = \beta_j X_j$; a GAM keeps the additive structure but frees each term to bend into a smooth non-linearity — capturing curved effects a straight line would miss.

One step further

The same idea extends to classification: place the additive predictor inside a link, e.g. $\log \frac{p(X)}{1-p(X)} = \beta_0 + \sum_j f_j(X_j)$ for a logistic GAM.

Problem 2 — solution (b): advantage and limitation

Solution P2 — (b) interpretability vs. interactions

Advantage (interpretability). Because the model is additive, the effect of each X_j on Y is captured by its own function f_j , which can be *plotted and read one panel at a time* while the other predictors are held fixed — non-linear effects stay as readable as a coefficient table.

Limitation. Additivity excludes *interactions*: the effect of X_j cannot depend on the value of X_k .

Repair. Add interaction components by hand — a product term $X_j \times X_k$, or a low-dimensional joint smooth $f_{jk}(X_j, X_k)$ as an extra component.

Problem 2 — solution (c): backfitting

Solution P2 — (c) the backfitting algorithm

Fit the f_j one at a time, cycling through $j = 1, \dots, p$. At each step update f_j by smoothing the *partial residuals* — what remains after removing all the *other* current terms:

$$r_i = y_i - \beta_0 - \sum_{k \neq j} f_k(x_{ik}), \quad \text{then re-fit } f_j \text{ to } (x_{ij}, r_i).$$

Every other function is held fixed while f_j is updated; repeat the whole cycle until the f_j stop changing (convergence).

Why it is convenient

Backfitting only ever needs a *one-dimensional* smoother, so any scatterplot smoother (spline, loess) can be plugged in — no giant multivariate fit required.

Problem 3 — task

Problem 3 (20 P) — Trees by hand

Advertising budget x (1,000 EUR) and weekly sales y (1,000 units) for $n = 6$ products; one split at s : $R_1 = \{x < s\}$ and $R_2 = \{x \geq s\}$ with region-mean predictions.

i	1	2	3	4	5	6
x_i	1	2	3	4	5	6
y_i	2	4	3	3	8	10

(a) **[8 P]** Evaluate the cutpoints $s = 2.5$ and $s = 4.5$ by total RSS; which is chosen? (b) **[2 P]** Using the chosen split $s = 4.5$ from (a) ($R_1 = \{x < 4.5\}$, $\hat{y}_{R_1} = 3$; $R_2 = \{x \geq 4.5\}$, $\hat{y}_{R_2} = 9$), prediction for a new product with $x = 5$. (c) **[6 P]** Node with 6 “Yes” / 2 “No”: proportions; Gini; entropy ($\ln 0.75 = -0.2877$; $\ln 0.25 = -1.3863$); predicted class. (d) **[4 P]** Cost-complexity pruning: role of α in $\sum_m \sum_{i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|$.

Lecture slides: Chapter 8 — Tree-Based Methods (Lecture 10).

Problem 3 — solution (a.1): root node and cutpoint $s = 2.5$

Solution P3 — (a) part 1: root node and cutpoint 2.5

Root node (no split), for reference. $\bar{y} = \frac{2+4+3+3+8+10}{6} = \frac{30}{6} = 5$, and

$$\text{RSS}_{\text{root}} = (2-5)^2 + (4-5)^2 + (3-5)^2 + (3-5)^2 + (8-5)^2 + (10-5)^2 = 9+1+4+4+9+25 = 52.$$

Cutpoint $s = 2.5$: $R_1 = \{1, 2\}$, $y \in \{2, 4\}$, $\hat{y}_{R_1} = 3$; $R_2 = \{3, 4, 5, 6\}$, $y \in \{3, 3, 8, 10\}$, $\hat{y}_{R_2} = \frac{24}{4} = 6$.

$$\text{RSS}_{R_1} = (2-3)^2 + (4-3)^2 = 2,$$

$$\text{RSS}_{R_2} = (3-6)^2 + (3-6)^2 + (8-6)^2 + (10-6)^2 = 9+9+4+16 = 38,$$

$$\implies \text{RSS}(2.5) = 2 + 38 = 40.$$

Problem 3 — solution (a.2): cutpoint $s = 4.5$ and the choice

Solution P3 — (a) part 2: cutpoint 4.5 and the choice

Cutpoint $s = 4.5$: $R_1 = \{1, 2, 3, 4\}$, $y \in \{2, 4, 3, 3\}$, $\hat{y}_{R_1} = \frac{12}{4} = 3$; $R_2 = \{5, 6\}$, $y \in \{8, 10\}$, $\hat{y}_{R_2} = 9$.

$$\text{RSS}_{R_1} = 1 + 1 + 0 + 0 = 2, \quad \text{RSS}_{R_2} = (8-9)^2 + (10-9)^2 = 2 \Rightarrow \text{RSS}(4.5) = 2 + 2 = 4.$$

s	$R_1 (y)$	$\hat{y}_{R_1}$	$R_2 (y)$	$\hat{y}_{R_2}$	total RSS
2.5	$\{2, 4\}$	3	$\{3, 3, 8, 10\}$	6	$2 + 38 = 40$
4.5	$\{2, 4, 3, 3\}$	3	$\{8, 10\}$	9	$2 + 2 = 4$

Decision. Recursive binary splitting is greedy and picks the smaller RSS: $4 < 40$, so it chooses $s = 4.5$ — isolating the four low-sales products (mean 3) from the two high-sales products (mean 9) and cutting the root RSS from 52 down to 4.

Problem 3 — solution (b): prediction for a new product

Solution P3 — (b) prediction for a new product

Drop the new observation down the tree using the chosen split $s = 4.5$:

$$x = 5 \geq 4.5 \implies \text{region } R_2 \implies \hat{y} = \hat{y}_{R_2} = 9$$

i.e. the tree predicts 9,000 units per week.

How a tree predicts

A regression tree is *piecewise constant*: every $x \in [4.5, 6]$ receives the same prediction $\hat{y}_{R_2} = 9$ (the region mean). Predictions change only by jumping at the split points.

Problem 3 — solution (c): Gini and entropy of a node

Solution P3 — (c) impurity of a 6-Yes / 2-No node

Class proportions: $\hat{p}_{\text{Yes}} = \frac{6}{8} = 0.75$, $\hat{p}_{\text{No}} = \frac{2}{8} = 0.25$.

$$G = \sum_k \hat{p}_k(1 - \hat{p}_k) = 0.75 \cdot 0.25 + 0.25 \cdot 0.75 = 0.1875 + 0.1875 = 0.375.$$

$$D = - \sum_k \hat{p}_k \ln \hat{p}_k = -(0.75 \ln 0.75 + 0.25 \ln 0.25)$$

$$= -(0.75 \cdot (-0.2877) + 0.25 \cdot (-1.3863)) = 0.2158 + 0.3466 = 0.562.$$

Predicted class: the majority, **Yes** (estimated probability 0.75).

Reading the numbers

Both G and D equal 0 for a pure node and are largest at a 50:50 split; here they sit in the middle, so this node is *moderately impure*.

Problem 3 — solution (d): cost-complexity pruning

Solution P3 — (d) cost-complexity pruning

In $\sum_m \sum_{i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|$ the parameter $\alpha \geq 0$ is the *price charged per terminal node* $|T|$: it trades the subtree's fit against its size.

- $\alpha = 0$: the criterion is just the training RSS $\Rightarrow$ the full tree T_0 wins (nothing penalises size).
- α increasing: each leaf must “pay for itself”; a branch whose RSS reduction is smaller than α is pruned. Growing α yields a *nested sequence* of ever-smaller subtrees (weakest-link pruning).
- Choosing α : by *cross-validation* — take the α with the smallest CV error, then refit that subtree on all the data.

Bias–variance

α is a complexity dial: small α = large low-bias/high-variance tree; large α = small high-bias/low-variance tree. CV locates the sweet spot.

Problem 4 — task

Problem 4 (16 P) — Ensembles

- (a) [6 P] B identically distributed trees with variance σ^2 and pairwise correlation ρ :
 $\text{Var}\left(\frac{1}{B} \sum_b \hat{f}_b(x)\right) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$. (i) Why does this show variance reduction and what limits it? (ii) Evaluate for $B = 100$ and $\rho = 0.6$ and $\sigma^2 = 4$; compare with one tree. (iii) Limit as $B \rightarrow \infty$?
- (b) [4 P] Random forests: modification at each split; recommended m for $p = 25$; why does it improve on bagging (use $\text{Var} = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$ from (a))?
- (c) [4 P] Boosting via `GradientBoostingRegressor(n_estimators=5000, learning_rate=0.01, max_depth=1)`: what does each of the three arguments control? The slow-learning idea (and how the chosen values reflect it)?
- (d) [2 P] What does the OOB error estimate and why is it free?

Lecture slides: Chapter 8 — Tree-Based Methods (Lecture 10).

Problem 4 — solution (a.1): why bagging reduces variance

Solution P4 — (a)(i) reading the variance formula

$$\text{Var}\left(\frac{1}{B} \sum_{b=1}^B \hat{f}_b(x)\right) = \underbrace{\rho \sigma^2}_{\text{correlated part}} + \underbrace{\frac{1-\rho}{B} \sigma^2}_{\text{independent part}} .$$

- The second term $\frac{1-\rho}{B} \sigma^2 \rightarrow 0$ as B grows: averaging cancels the *independent* part of the trees' variability — this is the variance reduction of bagging.
- The first term $\rho \sigma^2$ does *not* depend on B . Bootstrap trees are grown on overlapping samples of the same data and share the same strong predictors, so they are positively correlated; that correlated part cannot be averaged away and *limits* the reduction.

Problem 4 — solution (a.2): evaluate and take $B \rightarrow \infty$

Solution P4 — (a)(ii)–(iii) the numbers and the limit

(ii) $B = 100$, $\rho = 0.6$, $\sigma^2 = 4$:

$$\text{Var} = 0.6 \cdot 4 + \frac{1 - 0.6}{100} \cdot 4 = 2.4 + \frac{0.4 \cdot 4}{100} = 2.4 + 0.016 = 2.416,$$

versus $\sigma^2 = 4$ for a single tree — a reduction of about 40%.

(iii) $B \rightarrow \infty$: the independent term dies and the variance approaches the floor

$$\text{Var} \longrightarrow \rho\sigma^2 = 0.6 \cdot 4 = 2.4.$$

More trees never overfit

Raising B can only push the variance from 2.416 down toward 2.4; extra trees merely stabilise the average. To go *below* 2.4 you must lower ρ — exactly what random forests do.

Problem 4 — solution (b): random forests

Solution P4 — (b) decorrelating the trees

Modification. At *each split*, only a fresh *random subset* of m of the p predictors is allowed as splitting candidates.

Recommended m (classification): $m \approx \sqrt{p}$, so for $p = 25$: $m = \sqrt{25} = 5$.

Why it beats bagging. With bagging a dominant predictor tops the first split of almost every tree $\Rightarrow$ the trees look alike $\Rightarrow \rho$ is high. Withholding it from $\approx \frac{p-m}{p}$ of the split searches *decorrelates* the trees, lowering ρ — and by (a) a smaller ρ lowers the irreducible floor $\rho\sigma^2$ that averaging alone can never touch.

Problem 4 — solution (c.1): the three boosting parameters

Solution P4 — (c) what each constructor argument controls

```
GradientBoostingRegressor(n_estimators=5000,  
                           learning_rate=0.01, max_depth=1)
```

- $n_estimators = B$ — **number of trees** added sequentially; unlike bagging, boosting *can* overfit if B is too large, so choose B by CV (here 5000).
- $learning_rate = \lambda$ — **shrinkage**: scales each new tree's contribution, i.e. the speed of learning; smaller λ needs larger B (here 0.01).
- $max_depth = d$ — **splits per tree** (interaction depth): the complexity and interaction order of each tree; often $d = 1$ stumps suffice (here 1).

Problem 4 — solution (c.2): the “slow learning” idea

Solution P4 — (c) why boosting learns slowly

Boosting builds the model *sequentially*: each new tree is fitted to the current *residuals* (what the model still gets wrong), and only the small fraction λ of that tree is added,

$$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^{\text{new}}(x).$$

So the fit improves gradually and concentrates effort exactly where errors remain, instead of fitting the data hard in one step — which tends to generalise better.

The chosen values encode this: a *small* `learning_rate= 0.01` with *many* trees (`n_estimators= 5000`) and shallow `max_depth= 1` stumps.

Problem 4 — solution (d): out-of-bag error

Solution P4 — (d) what OOB estimates and why it is free

Each bootstrap sample is drawn with replacement, so on average about $\frac{1}{3}$ of the observations are left out — they are *out-of-bag* (OOB) for that tree.

Predict each observation using *only* the trees that never saw it, then aggregate: the resulting error is the **OOB error**, a valid estimate of the model's *test error*.

Why free: it reuses the bootstrap as a by-product — no separate validation set and no extra cross-validation loop are required.

Handy consequence

For large B the OOB error is essentially the same as LOOCV, but obtained in a single training run.

Problem 5 — task

Problem 5 (20 P) — Neural networks by hand

Input $x = (1, 2)$; hidden ReLU units $g(z) = \max\{0, z\}$; linear output:

$$A_1 = g(-1 + 1x_1 + 2x_2) \quad A_2 = g(-3 - 2x_1 + 1x_2) \quad f(x) = 1 + 2A_1 + 4A_2$$

- (a) **[8 P]** Forward pass step by step: pre-activations; A_1 and A_2 ; output $f(x)$. What did the ReLU do to unit 2 and why are non-linear activations essential?
- (b) **[6 P]** Logits $z = (1.0, 0.0, -1.0)$; true class 1: all three softmax probabilities and the cross-entropy loss ($e^1 = 2.718$; $e^0 = 1.000$; $e^{-1} = 0.368$; $\ln 4.086 = 1.408$).
- (c) **[4 P]** Parameter count of a $p = 4 \rightarrow k = 3 \rightarrow K = 2$ network with biases (show both layers).
- (d) **[2 P]** Conv layer: 32×32 input; $F = 5$; $P = 2$; $S = 1$: output size via $(W - F + 2P)/S + 1$.

Lecture slides: Chapter 10 — Deep Learning (Lecture 11).

Problem 5 — solution (a): forward pass

Solution P5 — (a) the forward pass

Pre-activations (weighted sum + bias):

$$z_1 = -1 + 1 \cdot 1 + 2 \cdot 2 = -1 + 1 + 4 = 4, \quad z_2 = -3 - 2 \cdot 1 + 1 \cdot 2 = -3 - 2 + 2 = -3.$$

ReLU activations $g(z) = \max\{0, z\}$:

$$A_1 = \max\{0, 4\} = 4, \quad A_2 = \max\{0, -3\} = 0.$$

Output (linear unit):

$$f(x) = 1 + 2A_1 + 4A_2 = 1 + 2 \cdot 4 + 4 \cdot 0 = 9.$$

What the ReLU did / why it matters

Unit 2 is *switched off* ($z_2 < 0 \Rightarrow A_2 = 0$) and adds nothing here (it could for other inputs). Without a non-linear g the network collapses into one linear function of x — the non-linearity is what buys flexibility.

Problem 5 — solution (b): softmax and cross-entropy

Solution P5 — (b) softmax and cross-entropy

Exponentiate the logits and sum:

$$e^{1.0} = 2.718, \quad e^{0.0} = 1.000, \quad e^{-1.0} = 0.368 \Rightarrow \sum = 2.718 + 1.000 + 0.368 = 4.086.$$

Softmax probabilities:

$$\hat{p}_1 = \frac{2.718}{4.086} = 0.665, \quad \hat{p}_2 = \frac{1.000}{4.086} = 0.245, \quad \hat{p}_3 = \frac{0.368}{4.086} = 0.090 \quad (\text{sum} = 1.000 \checkmark).$$

Cross-entropy for the true class 1:

$$-\ln \hat{p}_1 = -\ln \frac{e^1}{4.086} = -(1 - \ln 4.086) = 1.408 - 1 = 0.408.$$

Sanity check

A perfect $\hat{p}_1 = 1$ gives loss 0; the uniform guess $\frac{1}{3}$ gives $\ln 3 \approx 1.099$. Our 0.408 sits well below — the net is right and fairly confident.

Problem 5 — solution (c): parameter count

Solution P5 — (c) parameters of a $4 \rightarrow 3 \rightarrow 2$ network

Every hidden and output unit has a bias, so each layer contributes (inputs $\times$ units) weights plus (units) biases:

Layer	Weights	Biases	Subtotal
input $\rightarrow$ hidden	$4 \cdot 3 = 12$	3	15
hidden $\rightarrow$ output	$3 \cdot 2 = 6$	2	8
Total			23

General rule

A dense layer from a to b units has $a \cdot b + b = b(a+1)$ parameters. Summing the two layers: $3(4+1) + 2(3+1) = 15 + 8 = 23$.

Problem 5 — solution (d): convolution output size

Solution P5 — (d) output of a convolutional layer

With input width $W = 32$, filter size $F = 5$, padding $P = 2$ and stride $S = 1$:

$$\frac{W - F + 2P}{S} + 1 = \frac{32 - 5 + 2 \cdot 2}{1} + 1 = \frac{31}{1} + 1 = 32.$$

The height is computed identically, so the output feature map is 32×32 (one such map per filter).

“Same” convolution

Here $P = (F - 1)/2$ with $S = 1$, which preserves the spatial size — a “same” convolution. A larger stride $S > 1$ or less padding would shrink the feature map.

Problem 6 — task

Problem 6 (20 P) — Multiple testing

- (a) [4 P] $m = 20$ independent tests at $\alpha = 0.05$; all nulls true: define and compute the FWER ($(0.95)^{20} = 0.3585$); interpret.
- (b) [7 P] Six ordered p -values 0.001; 0.004; 0.011; 0.030; 0.045; 0.230 — FWER control at $\alpha = 0.05$ with (i) Bonferroni and (ii) Holm; thresholds; rejected hypotheses; why is Holm never weaker than Bonferroni?
- (c) [6 P] Benjamini–Hochberg at $q = 0.05$ on the same six p -values from (b) (0.001; 0.004; 0.011; 0.030; 0.045; 0.230): thresholds jq/m ; rejections; what exactly does FDR control guarantee?
- (d) [3 P] Exploratory screen of 20,000 genes with follow-up validation: FWER or FDR? Justify.

Lecture slides: Chapter 13 — Multiple Testing (Lecture 12).

Problem 6 — solution (a): FWER of 20 unadjusted tests

Solution P6 — (a) family-wise error rate

Definition. $\text{FWER} = \Pr(V \geq 1)$, the probability of *at least one* false rejection, where V counts the true nulls that are wrongly rejected.

Compute (all 20 nulls true, tests independent, each at level 0.05):

$$\text{FWER} = 1 - \Pr(\text{no false rejection}) = 1 - (1 - 0.05)^{20} = 1 - (0.95)^{20} = 1 - 0.3585 = 0.6415.$$

Interpretation

Each test errs only 5% of the time, yet there is a $\approx 64\%$ chance of at least one spurious “discovery” among the 20. With $m = 100$ it is $1 - 0.95^{100} \approx 0.994$ — unadjusted multiple testing all but guarantees false findings.

Problem 6 — solution (b.1): Bonferroni

Solution P6 — (b) the Bonferroni correction

Rule. Reject $H_{(j)}$ iff $p_{(j)} \leq \frac{\alpha}{m} = \frac{0.05}{6} = 0.00833$ — the *same* threshold for every j .

j	1	2	3	4	5	6
$p_{(j)}$	0.001	0.004	0.011	0.030	0.045	0.230
$\leq 0.00833?$	✓	✓	×	×	×	×

Only 0.001 and 0.004 clear the bar ($0.011 > 0.00833$) $\Rightarrow$ reject $H_{(1)}$ and $H_{(2)}$: **2 rejections**.

Problem 6 — solution (b.2): Holm step-down

Solution P6 — (b) the Holm step-down procedure

Rule. Step down $j = 1, 2, \dots$, comparing $p_{(j)}$ with $\frac{\alpha}{m-j+1}$; reject while $p_{(j)}$ stays below its threshold and *stop at the first failure*.

j	1	2	3	4	5	6
$p_{(j)}$	0.001	0.004	0.011	0.030	0.045	0.230
$\alpha/(m-j+1)$	0.00833	0.0100	0.0125	0.0167	0.0250	0.0500
decision	✓	✓	✓	stop	—	—

$0.011 \leq 0.0125$ but $0.030 > 0.0167 \Rightarrow$ reject $H_{(1)}, H_{(2)}, H_{(3)}$: **3 rejections**.

Holm $\supseteq$ Bonferroni

Holm's hurdles satisfy $\alpha/(m-j+1) \geq \alpha/m$ for every j (equal only at $j = 1$), so Holm rejects *at least* whatever Bonferroni does — uniformly more powerful, and still valid without any independence assumption.

Problem 6 — solution (c.1): Benjamini–Hochberg

Solution P6 — (c) the Benjamini–Hochberg procedure

Rule. Compare $p_{(j)}$ with $\frac{j\alpha}{m} = \frac{j \cdot 0.05}{6}$; let L be the *largest* j that passes, then reject $H_{(1)}, \dots, H_{(L)}$.

j	1	2	3	4	5	6
$p_{(j)}$	0.001	0.004	0.011	0.030	0.045	0.230
$j\alpha/m$	0.00833	0.0167	0.0250	0.0333	0.0417	0.0500
$p_{(j)} \leq j\alpha/m?$	yes	yes	yes	yes	no	no

Largest passing index: $L = 4$ ($0.030 \leq 0.0333$, while $0.045 > 0.0417$). BH rejects *all* of $H_{(1)}, \dots, H_{(4)}$: **4 rejections** — note $H_{(4)}$ is rejected even though on its own 0.030 is not tiny.

Problem 6 — solution (c.2): what FDR control guarantees

Solution P6 — (c) the FDR guarantee

$$\text{FDR} = \mathbb{E} \left[\frac{V}{\max(R, 1)} \right] \leq q = 0.05,$$

where R is the number of rejections and V the number of *false* ones. In words: *on average* at most 5% of the hypotheses BH rejects are false discoveries.

It does *not* promise that at most 5% of *these particular* 4 rejections are false, nor does it bound $\Pr(V \geq 1)$ — that stronger guarantee would be FWER control.

Same data — three procedures

Bonferroni $\rightarrow$ 2, Holm $\rightarrow$ 3, BH $\rightarrow$ 4 rejections. Controlling the milder FDR instead of the FWER buys more power (more discoveries) at the cost of a few expected false positives.

Problem 6 — solution (d): FWER or FDR for a genomics screen?

Solution P6 — (d) exploratory screen of 20000 genes

Recommendation: control the FDR.

- FWER control (e.g. Bonferroni at $\frac{0.05}{20,000} = 2.5 \times 10^{-6}$) guards against even a *single* false positive
⇒ almost no power; nearly every true signal would be missed.
- The study is *exploratory* and every hit is re-tested in an independent follow-up, so a few false leads are cheap — what matters is that the candidate *list* is mostly reliable.
- FDR control at $q = 0.05$ – 0.2 delivers exactly that: a rich list of candidates of which, on average, only a known small fraction is false.

Problem 7 — task

Problem 7 (20 P) — Cumulative essentials

- (a) [4 P] True or false: “more flexible always means lower *test* error”. Justify.
- (b) [4 P] Logistic regression for default in `statsmodels`; the `sm.Logit` output gives `coef = 0.7` for `missed_payments` ($e^{0.7} = 2.014$): effect of one more missed payment on the *odds*; why is there no single number for the effect on the *probability*?
- (c) [4 P] One reason to prefer 5- or 10-fold CV over LOOCV and one over a single validation split.
- (d) [2 P] $p = 2,000$ predictors; only a handful truly matter: ridge or lasso? Why?
- (e) [6 P] Match to {linear regression; lasso; logistic regression; random forest} — each exactly once: (i) price per m^2 with CI; (ii) drug response from 5,000 genes ($n = 100$) with a short gene list; (iii) default yes/no with odds reported to the regulator; (iv) machine failures from hundreds of interacting sensors; accuracy only.

Lecture slides: Chapters 2–6 (Lectures 1–8).

Problem 7 — solution (a): flexibility and test error

Solution P7 — (a) “more flexible $\Rightarrow$ lower test error”?

False. The expected test error decomposes as

$$\mathbb{E}(y_0 - \hat{f}(x_0))^2 = \underbrace{\text{Var}(\hat{f}(x_0))}_{\uparrow \text{ with flexibility}} + \underbrace{[\text{Bias}(\hat{f}(x_0))]^2}_{\downarrow \text{ with flexibility}} + \underbrace{\text{Var}(\varepsilon)}_{\text{irreducible}} .$$

Flexibility lowers bias but raises variance; past the sweet spot the variance increase wins and the test error *rises* — the characteristic U-shape. E.g. when the true f is nearly linear or n is small, a flexible method overfits and a simple linear fit has *lower* test error.

The one thing that is monotone

Only the *training* error falls monotonically with flexibility; test error is U-shaped — which is exactly why we *estimate* it (CV) rather than trust training fit.

Problem 7 — solution (b): odds vs. probability

Solution P7 — (b) interpreting the logistic coefficient 0.7

From `sm.Logit`, $\hat{\beta}_1 = 0.7$ for `missed_payments` (highly significant, $z = 4.667$). One more missed payment adds 0.7 to the *log-odds*, i.e. *multiplies the odds* of default by

$$e^{\hat{\beta}_1} = e^{0.7} = 2.014$$

— the odds roughly *double*, by the same factor at every value of x .

Why no single number for the probability. The effect on $p(x)$ depends on where the applicant sits on the S-shaped logistic curve: a 0.7 log-odds shift moves p a lot near $p \approx 0.5$ but hardly at all when p is close to 0 or 1.

Problem 7 — solution (c): why 5- or 10-fold CV

Solution P7 — (c) k -fold CV vs. LOOCV and vs. one split

vs. LOOCV. The n leave-one-out fits are trained on almost identical data sets, so their errors are highly positively correlated; averaging strongly correlated quantities reduces variance little, so the LOOCV estimate has *higher variance* (and costs n fits rather than k).

vs. a single validation split. One split is highly variable and *overestimates* the test error, because only part of the data trains the model (bias). k -fold CV trains on $\frac{k-1}{k}$ of the data and averages k estimates $\Rightarrow$ less bias and more stability.

Sweet spot

$k = 5$ or 10 is the usual compromise: much cheaper and lower-variance than LOOCV, yet far less biased and less noisy than a single hold-out split.

Problem 7 — solution (d): ridge or lasso?

Solution P7 — (d) ridge versus lasso

Lasso. The ℓ_1 penalty can set coefficients *exactly to zero*, so it performs *variable selection* — ideal when only a handful of the 2,000 predictors truly matter, and it returns a small, interpretable model.

Why not ridge. The ℓ_2 penalty only shrinks coefficients; it keeps all 2,000 predictors in the model and tends to lose when most true coefficients are zero.

Rule of thumb

Sparse truth $\rightarrow$ lasso (selection); many small effects that all contribute $\rightarrow$ ridge (shrinkage). The elastic net blends the two.

Problem 7 — solution (e): match method to scenario

Solution P7 — (e) each method used exactly once

- (i) **Linear regression** — quantitative response, goal is *inference*: the effect of one extra m^2 with a confidence interval, straight from OLS standard errors.
- (ii) **Lasso** — $p = 5,000 \gg n = 100$ makes OLS infeasible; the ℓ_1 penalty regularises and returns a *short list* of relevant genes.
- (iii) **Logistic regression** — binary response (default yes/no) with interpretable coefficients $e^{\hat{\beta}_j} = \text{odds ratios}$, suitable for the regulator.
- (iv) **Random forest** — captures non-linearities and interactions among hundreds of mixed sensors automatically; high accuracy, interpretability not required.